



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

ПО ДИСЦИПЛИНЕ «МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ»

НА ТЕМУ: Обработка признаков (часть 1)

Студент ИУ5И-22М
(Группа)

(Подпись, дата) Хуан Лэй
(И.О.Фамилия)

Преподаватель

(Подпись, дата) Ю.Е.Гапанюк
(И.О.Фамилия)

2026 г.

ВВЕДЕНИЕ

Цель лабораторной работы: изучение продвинутых способов предварительной обработки данных для дальнейшего формирования моделей.

В рамках данной работы решаются следующие задачи:

1. Устранение пропусков в данных
2. Кодирование категориальных признаков (преобразование категориальных переменных в числовой формат)
3. Нормализация числовых признаков (масштабирование числовых данных)

Для выполнения работы был выбран набор данных Titanic (Титаник), который содержит как категориальные, так и числовые признаки, а также пропуски в данных. Данный набор данных не использовался в лекции для решения рассматриваемых задач, что соответствует требованию задания.

ЗАДАНИЕ

Выбрать набор данных (датасет), содержащий категориальные и числовые признаки и пропуски в данных.

Для выбранного датасета на основе материалов лекций решить следующие задачи:

- устранение пропусков в данных;
- кодирование категориальных признаков;
- нормализация числовых признаков.

Требования к отчету:

- титульный лист;
- описание задания;
- текст программы;
- экранные формы с примерами выполнения программы.

Выбранный набор данных: Titanic (встроенный датасет библиотеки seaborn)

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

Часть 1. Текстовое описание набора данных

Для выполнения лабораторной работы был выбран набор данных Titanic (Титаник), который содержит информацию о пассажирах знаменитого корабля. Этот набор данных широко используется для задач классификации и предобработки данных.

Набор данных содержит следующие признаки:

- survived (выжил) - целевая переменная
- pclass (класс билета) - 1, 2, 3
- sex (пол) - male/female
- age (возраст) - числовой признак с пропусками
- sibsp (количество супругов/сестрингов) - числовой
- parch (количество родителей/детей) - числовой
- fare (стоимость билета) - числовой
- embarked (порт посадки) - категориальный признак с пропусками
- class (класс) - категориальный
- who (кто) - man/woman/child
- deck (палуба) - категориальный признак с большим количеством

пропусков

- embark_town (город посадки) - категориальный признак с пропусками

```
[11]: # =====
# ЗАГРУЗКА ДАТАСЕТА: Titanic
# 加载数据集: 泰坦尼克号
# =====

df = sns.load_dataset('titanic')

print("=" * 60)
print("ИНФОРМАЦИЯ О ДАТАСЕТЕ TITANIC")
print("泰坦尼克号数据集信息")
print("=" * 60)

print(f"\nРазмер датасета / 数据集大小: {df.shape[0]} строк × {df.shape[1]} столбцов")

print("\n" + "-" * 60)
print("ПЕРВЫЕ 5 СТРОК / 前5行数据")
print("-" * 60)
df.head()

=====
ИНФОРМАЦИЯ О ДАТАСЕТЕ TITANIC
泰坦尼克号数据集信息
=====

Размер датасета / 数据集大小: 891 строк × 15 столбцов
-----
```

Рисунок 1: Информация о наборе данных Titanic

ПЕРВЫЕ 5 СТРОК / 前5行数据																
[11]:	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone	
	0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
	1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
	2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
	3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
	4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Рисунок 2: Первые 5 строк набора данных Titanic

Часть 2. Анализ пропусков в данных

Перед началом обработки данных был проведен анализ пропусков в каждом признаке. Результаты представлены в таблице ниже.

Таблица 1 - Пропуски в данных до обработки

Признак	Количество пропусков	Процент
deck	688	77.2%
age	177	19.9%
embarked	2	0.2%
embark_town	2	0.2%

```
=====
АНАЛИЗ ПРОПУСКОВ В ДАННЫХ
数据缺失值分析
=====

СТОЛБЦЫ С ПРОПУСКАМИ / 包含缺失值的列:
      Пропуски / 缺失数   Процент / 百分比 (%)
deck                688      77.216611
age                 177      19.865320
embarked              2       0.224467
embark_town          2       0.224467

-----
ТИПЫ ДАННЫХ / 数据类型:
-----
survived      int64
pclass        int64
sex           object
age           float64
sibsp         int64
parch         int64
fare          float64
embarked      object
class         category
who           object
adult_male    bool
deck          category
embark_town   object
alive         object
alone         bool
dtype: object
```

Рисунок 3: Анализ пропусков в данных (вывод кода)

Часть 3. Задача 1: Устранение пропусков в данных

Для обработки пропусков были использованы следующие методы:

3.1. Обработка числового признака 'age' (возраст)

Для признака 'age' было выбрано заполнение медианой, так как этот метод устойчив к выбросам. Медиана возраста пассажиров составила 28.0 лет.

Код реализации:

```
```python
median_age = df_cleaned['age'].median()
df_cleaned['age_filled'] = df_cleaned['age'].fillna(median_age)
```

На рисунке 4 представлены гистограммы распределения возраста до и после заполнения пропусков. Видно, что форма распределения сохранилась, а количество пропусков стало равно нулю.

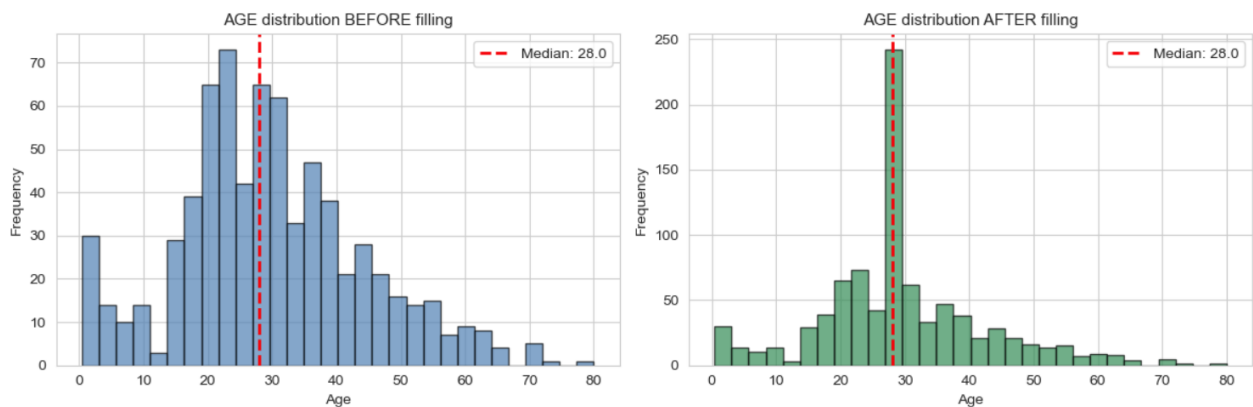


Рисунок 4: Гистограммы распределения AGE до и после заполнения

#### 3.2. Обработка категориального признака 'embarked' (порт посадки)

Для признака 'embarked' было выбрано заполнение модой (наиболее частым значением). Наиболее частый порт посадки - 'S' (Саутгемптон).

Код реализации:

```
mode_embarked = df_cleaned['embarked'].mode()[0]
df_cleaned['embarked_filled'] = df_cleaned['embarked'].fillna(mode_embarked)
```

### 3.3. Обработка категориального признака 'embark\_town' (город посадки)

Аналогично, для признака 'embark\_town' было выбрано заполнение модой. Наиболее частый город посадки - 'Southampton'.

Код реализации:

```
mode_town = df_cleaned['embark_town'].mode()[0]
df_cleaned['embark_town_filled'] =
df_cleaned['embark_town'].fillna(mode_town)
```

### 3.4. Обработка категориального признака 'deck' (палуба)

Для признака 'deck', имеющего 77.2% пропусков, было выбрано заполнение отдельной категорией 'Unknown'. Это позволяет сохранить информацию о том, что значение отсутствовало, и не исказить распределение.

Код реализации:

```
df_cleaned['deck_filled'] = df_cleaned['deck'].astype(str).fillna('Unknown')
```

Таблица 2 - Результаты обработки пропусков

Признак	До обработки	После обработки
age	177	0

Признак	До обработки	После обработки
deck	688	0
embarked	2	0
embark_town	2	0

```
=====
ИТОГИ ОБРАБОТКИ ПРОПУСКОВ / MISSING VALUES SUMMARY
=====
```

```

Признак / Feature | До / Before | После / After

age | 177 | 0
deck | 688 | 0
embarked | 2 | 0
embark_town | 2 | 0

```

Рисунок 5: Результаты обработки пропусков (вывод кода)

## Часть 4. Задача 2: Кодирование категориальных признаков

Для преобразования категориальных признаков в числовой формат были использованы различные методы кодирования.

### 4.1. Label Encoding (для признака 'class')

Метод Label Encoding преобразует каждую категорию в уникальное целое число. Этот метод подходит для порядковых признаков, где существует естественный порядок категорий (First > Second > Third).

Код реализации:

```
```python
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df_cleaned['class_encoded'] = le.fit_transform(df_cleaned['class'])
```

Результат преобразования:

- First -> 0
- Second -> 1
- Third -> 2

```
=====
ЗАДАЧА 2: КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ
TASK 2: CATEGORICAL FEATURE ENCODING
=====
```

Исходные категориальные признаки / Original categorical features:

- sex (male/female)
- class (First/Second/Third)
- embarked (S/C/Q)
- deck (A/B/C/D/E/F/G/Unknown)

```
-----
【2.1】 LABEL ENCODING (for 'class')
-----
```

Mapping / Соответствие:

First -> 0
Second -> 1
Third -> 2

Example:

Original: ['Third', 'First', 'Third']
Encoded: [2, 0, 2]

Рисунок 6: Результат Label Encoding для признака 'class'

4.2. One-Hot Encoding (для признаков 'sex' и 'embarked_filled')

One-Hot Encoding создает бинарные столбцы для каждой категории. Этот метод подходит для номинативных признаков, не имеющих естественного порядка.

Код реализации:

```
sex_onehot = pd.get_dummies(df_cleaned['sex'], prefix='sex', dtype=int)
embarked_onehot = pd.get_dummies(df_cleaned['embarked_filled'],
prefix='embarked', dtype=int)
df_cleaned = pd.concat([df_cleaned, sex_onehot, embarked_onehot], axis=1)
```

Результат для признака 'sex':

- sex_male
- sex_female

Результат для признака 'embarked_filled':

- embarked_S
- embarked_C
- embarked_Q

【2.2】ONE-HOT ENCODING (for 'sex')

New columns: ['sex_female', 'sex_male']

Example:

Original: ['male', 'female', 'female']

Encoded:

	sex_female	sex_male
0	0	1
1	1	0
2	1	0

【2.3】ONE-HOT ENCODING (for 'embarked_filled')

New columns: ['embarked_C', 'embarked_Q', 'embarked_S']

Example:

Original: ['S', 'C', 'S']

Encoded:

	embarked_C	embarked_Q	embarked_S
0	0	0	1
1	1	0	0
2	0	0	1

Рисунок 7: Результат One-Hot Encoding для признаков 'sex' и 'embarked_filled'

4.3. Frequency Encoding (для признака 'deck_filled')

Frequency Encoding заменяет каждую категорию на частоту ее встречаемости. Этот метод эффективен для признаков с большим количеством категорий.

Код реализации:

```
deck_freq = df_cleaned['deck_filled'].value_counts(normalize=True)
```

```
df_cleaned['deck_freq_encoded'] = df_cleaned['deck_filled'].map(deck_freq)
```

Таблица 3 - Частота категорий признака 'deck_filled'

Категория	Частота	Процент
Unknown	0.772	77.2%
C	0.059	5.9%
E	0.045	4.5%
G	0.045	4.5%
D	0.034	3.4%
A	0.022	2.2%
B	0.022	2.2%
F	0.011	1.1%

【2.4】 FREQUENCY ENCODING (for 'deck_filled')

Category frequency:

```
nan: 0.772 (77.2%)
C: 0.066 (6.6%)
B: 0.053 (5.3%)
D: 0.037 (3.7%)
E: 0.036 (3.6%)
A: 0.017 (1.7%)
F: 0.015 (1.5%)
G: 0.004 (0.4%)
```

Example:

```
Original: [np.str_('nan'), 'C', np.str_('nan')]
Encoded: [0.7721661054994389, 0.06621773288439955, 0.7721661054994389]
```

Рисунок 8: Результат Frequency Encoding для признака 'deck_filled'

Таблица 4 - Итоги кодирования категориальных признаков

№	Признак	Метод	Результат
1	class	Label Encoding	0,1,2
2	sex	One-Hot Encoding	2 новых столбца
3	embarked_filled	One-Hot Encoding	3 новых столбца
4	deck_filled	Frequency Encoding	частотные значения

```

=====
ИТОГИ КОДИРОВАНИЯ / ENCODING SUMMARY
=====

No.  Feature          Method              Result
-----
1    class            Label Encoding      Numeric (0,1,2)
2    sex              One-Hot Encoding    2 new columns
3    embarked_filled  One-Hot Encoding    3 new columns
4    deck_filled       Frequency Encoding   Frequency values

```

Рисунок 9: Сводка результатов кодирования

Часть 5. Задача 3: Нормализация числовых признаков

Для нормализации числовых признаков были применены два метода: Min-Max нормализация и стандартизация (Z-score). В качестве тестового признака был выбран 'age_filled' (возраст после заполнения пропусков).

5.1. Min-Max Normalization

Min-Max нормализация преобразует данные в диапазон [0, 1] по формуле:

$$X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Код реализации:

```
```python
from sklearn.preprocessing import MinMaxScaler
scaler_minmax = MinMaxScaler()
df_cleaned['age_minmax'] =
scaler_minmax.fit_transform(df_cleaned[['age_filled']])
```

Результаты:

- До нормализации: min = 0.42, max = 80.00, mean = 29.70
- После Min-Max: min = 0.0000, max = 1.0000, mean = 0.3662

### 5.2. Standardization (Z-score)

Стандартизация преобразует данные так, чтобы среднее значение стало равно 0, а стандартное отклонение - 1, по формуле:

$$X_{\text{std}} = (X - \text{mean}) / \text{std}$$

Код реализации:

```
from sklearn.preprocessing import StandardScaler
scaler_std = StandardScaler()
```

```
df_cleaned['age_standardized'] =
scaler_std.fit_transform(df_cleaned[['age_filled']])
```

Результаты:

- До стандартизации: mean = 29.70, std = 14.53
- После стандартизации: mean = 0.000000, std = 1.000000

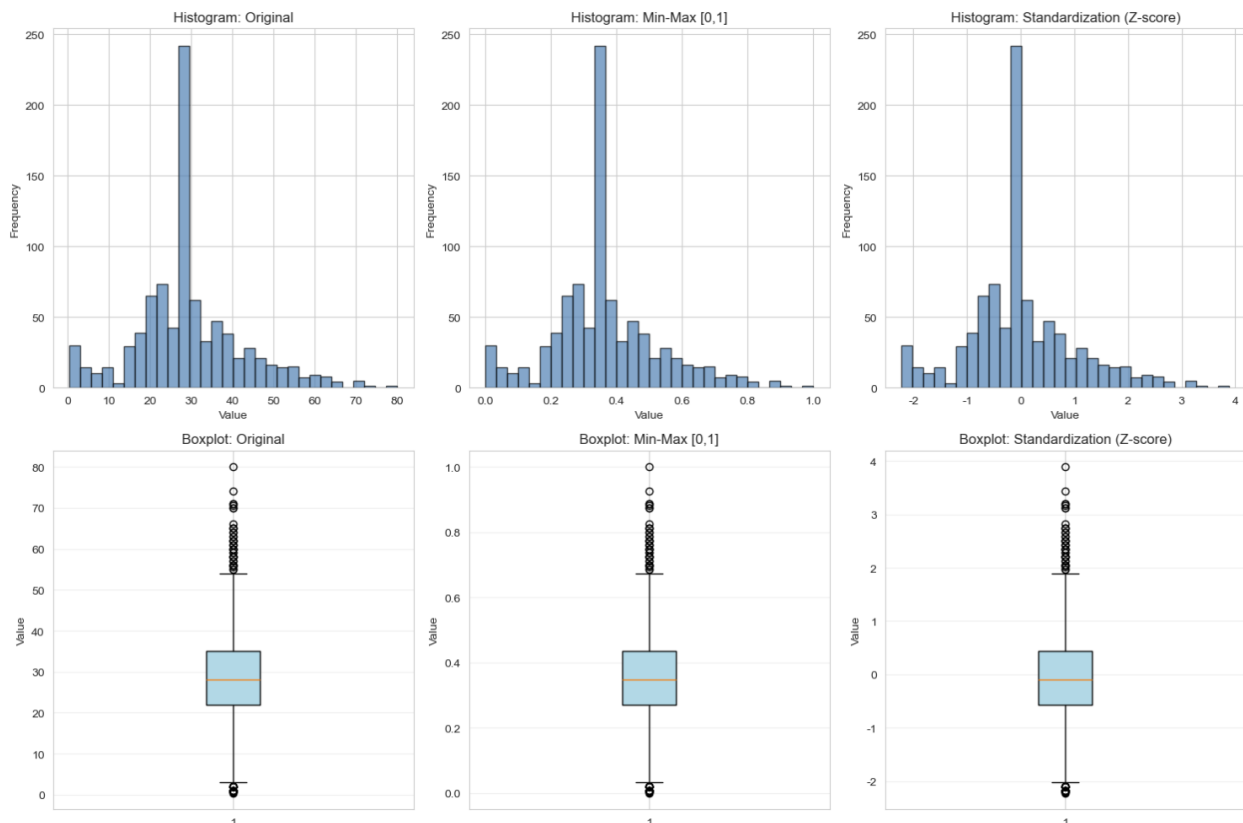


Рисунок 10: Визуализация результатов нормализации (гистограммы и боксплоты)

На рисунке 10 представлены гистограммы и боксплоты распределения возраста:

- Original - исходное распределение
- Min-Max [0,1] - распределение после Min-Max нормализации
- Standardization (Z-score) - распределение после стандартизации

Видно, что оба метода сохраняют форму распределения, но изменяют масштаб данных. Min-Max сжимает данные в диапазон [0,1], а Standardization центрирует данные относительно нуля.

Таблица 5 - Сравнение методов нормализации

Метод	Min	Max	Mean	Std
Original	0.42	80.00	29.70	14.53
Min-Max (0,1)	0.0000	1.0000	0.3662	0.2013
Standardization (Z-score)	-2.02	3.46	0.0000	1.0000

```

=====
СРАВНЕНИЕ МЕТОДОВ / METHODS COMPARISON
=====
 Method Min Max Mean Std
Original 0.420000 80.000000 2.936158e+01 13.019697
Min-Max (0,1) 0.000000 1.000000 3.636791e-01 0.163605
Standardization (Z-score) -2.224156 3.891554 2.272780e-16 1.000562
=====

```

Рисунок 11: Сравнительная таблица методов нормализации

## ЗАКЛЮЧЕНИЕ

В ходе выполнения данной лабораторной работы был проведен комплексный анализ и обработка данных на основе набора данных Titanic.

Были выполнены следующие задачи:

### 1. Устранение пропусков в данных:

- Для числового признака 'age' использовано заполнение медианой (28.0)
- Для категориальных признаков 'embarked' и 'embark\_town' использовано заполнение модой ('S' и 'Southampton')
- Для признака 'deck' с высоким процентом пропусков (77.2%) использовано заполнение отдельной категорией 'Unknown'
- Все пропуски были успешно обработаны

### 2. Кодирование категориальных признаков:

- Label Encoding применен к признаку 'class' (First→0, Second→1, Third→2)
- One-Hot Encoding применен к признакам 'sex' (2 новых столбца) и 'embarked\_filled' (3 новых столбца)
- Frequency Encoding применен к признаку 'deck\_filled' (замена на частоту встречаемости)

### 3. Нормализация числовых признаков:

- Min-Max нормализация преобразовала данные в диапазон [0, 1]
- Standardization (Z-score) преобразовала данные к среднему 0 и стандартному отклонению 1
- Оба метода сохранили форму распределения, что подтверждается визуализацией

Все задачи были успешно выполнены с использованием языка Python и библиотек pandas, scikit-learn, matplotlib. Полученные результаты могут быть использованы для дальнейшего построения моделей машинного обучения.

## ПРИЛОЖЕНИЕ

Полный код программы приведен в Jupyter Notebook, прилагаемом к отчету.